

Replikation und Konsistenz

Worum geht es?

In verteilten Systemen gibt es oft mehrere Kopien derselben Daten.
Dann stellt sich die Frage:

- Wie bleiben diese Kopien **konsistent**?
 - Welche Garantien bekommen Clients beim **Lesen und Schreiben**?
 - Was passiert bei **Netzwerkfehlern, Partitions** und **gleichzeitigen Zugriffen**?
-

1. Konsistenz: der Begriff ist mehrdeutig

„Konsistenz“ kann je nach Kontext etwas anderes bedeuten:

ACID-Konsistenz

Eine Transaktion überführt die Datenbank von einem gültigen Zustand in einen anderen gültigen Zustand.

Die Regeln dafür sind **anwendungsspezifisch**. Beispiel:

- Ausgaben dürfen das Budget nicht überschreiten.

Read-after-write Consistency

Wer etwas schreibt und direkt danach liest, soll den gerade geschriebenen Wert sehen.

Konsistenz zwischen Replikas

Mehrere Replika sollen denselben oder einen ausreichend ähnlichen Zustand haben.
Je nach Modell heißt das z. B.:

- gleicher Zustand
 - gleiche Leseergebnisse
 - nur irgendwann gleiche Daten
-

2. Verteilte Transaktionen

Motivation

Eine Aktion kann mehrere getrennte Systeme betreffen, zum Beispiel:

- Produktdatenbank
- Lagerverwaltung

Wenn ein Produkt verkauft wird, müssen beide Systeme konsistent aktualisiert werden. Dafür braucht man eine **verteilte Transaktion**.

Atomic Commitment

Bei verteilten Transaktionen gilt:

- Entweder **alle** beteiligten Knoten committen
- oder **keiner** commitet

Wenn ein beteiligter Knoten ausfällt, muss die Transaktion im Zweifel überall abgebrochen werden.

Das ist das **Atomic-Commitment-Problem**.

Unterschied zu Konsens

Atomic Commit und Konsens sehen ähnlich aus, sind aber nicht gleich:

Konsens

- Knoten einigen sich auf einen Wert
- Abstürze können toleriert werden, solange ein Quorum entscheidet

Atomic Commit

- Alle Transaktionsteilnehmer müssen committen können
 - Fällt ein Teilnehmer aus, muss abgebrochen werden
-

3. Serialisierbarkeit

Idee

Operationen sollen sich so verhalten, als würden sie in einer **seriellen Reihenfolge** nacheinander ablaufen.

Das ist die klassische Umsetzung des **Isolation**-Aspekts in ACID.

Bedeutung

Wenn jede serialisierte Transaktion das System von einem korrekten Zustand in den nächsten überführt, unterstützt das auch die fachliche Konsistenz.

4. Two-Phase Commit (2PC)

Rolle des Koordinators

Bei verteilten Transaktionen gibt es typischerweise einen **Koordinator**:

- startet die Transaktion
 - vergibt eine ID
 - sammelt Teilnehmer
 - entscheidet am Ende commit oder abort
-

Ablauf von 2PC

Phase 1: Abstimmung

Der Koordinator fragt alle Teilnehmer:

- Kannst du committen?

Teilnehmer antworten mit:

- **Yes**
- **No**

Phase 2: Entscheidung

Danach teilt der Koordinator mit:

- **doCommit**
- oder **doAbort**

Die Teilnehmer führen das aus und bestätigen es.

Problem von 2PC

Wenn der Koordinator nach der Prepare-Phase abstürzt, aber bevor er seine Entscheidung verschickt, dann wissen die Teilnehmer nicht, wie sie weitermachen sollen.

Folge:

- sie dürfen weder committen noch aborten
 - das System **blockiert**
 - der Koordinator ist ein **Single Point of Failure**
-

Fehlertolerantere Variante

Die Folien zeigen eine fehlertolerantere Form von 2PC, bei der Votes per **Total Order Broadcast** verteilt werden.

Die Grundidee:

- alle sehen dieselben Abstimmungen in derselben Reihenfolge
 - commit nur, wenn alle mit Ja stimmen
 - sonst abort
-

5. Linearisierbarkeit

Grundidee

Das stärkste hier betrachtete Modell ist **Linearisierbarkeit**.

Informell heißt das:

- jede Operation wirkt so, als würde sie **atomar**
- zu einem einzigen Zeitpunkt
- zwischen Aufruf und Rückgabe stattfinden

Konsequenz

Das System verhält sich so, als gäbe es nur **eine einzige Kopie** der Daten.

Jede Operation sieht also den **aktuellsten** Wert.

Wichtig

Linearisierbarkeit ist nicht dasselbe wie Serialisierbarkeit.

6. Read-after-write ist schwächer als Linearisierbarkeit

Read-after-write garantiert nur:

- derselbe Client sieht nach seinem Write seinen Wert

Aber Linearisierbarkeit fordert mehr:

- auch **andere Clients** müssen passende aktuelle Werte sehen
- und zwar in einer Reihenfolge, die mit der realen Zeit verträglich ist

7. Quoren reichen nicht automatisch für Linearisierbarkeit

Auch mit Quorum-Reads und Quorum-Writes kann es passieren:

- Client 2 liest schon den neuen Wert
- Client 3 liest später trotzdem noch den alten Wert

Dann ist das System **nicht linearisiert**, obwohl Quoren verwendet wurden.
Das zeigen die Diagramme auf den Folien explizit.

8. ABD-Algorithmus

Idee

Der **ABD-Algorithmus** macht Quorum-Reads und Quorum-Writes linearisierbar.

Wichtiger Punkt

Eine Leseoperation ist erst fertig, wenn der gelesene aktuelle Wert wieder auf eine Mehrheit der angefragten Replika zurückgeschrieben wurde, falls dort Inkonsistenzen gefunden wurden.

Wirkung

Danach sehen spätere Quorum-Operationen ebenfalls diesen aktuellen Wert.

9. Linearisierbares Compare-and-Swap (CAS)

Eine Operation wie

```
CAS(x, oldValue, newValue)
```

liest und schreibt logisch atomar.

Das kann in verteilten Systemen linearisierbar implementiert werden, wenn man **Total Order**

Broadcast nutzt:

- alle Operationen werden total geordnet verteilt
 - sie werden erst bei Lieferung ausgeführt
 - dadurch führen alle Replika dieselben Operationen in derselben Reihenfolge aus
-

10. Strict Serializability

Bedeutung

Wenn ein System gleichzeitig erfüllt:

- **Linearisierbarkeit**
- **Serialisierbarkeit**

dann spricht man von **Strict Serializability** oder **External Consistency**.

Einordnung

Das ist der „Gold Standard“ der Konsistenz in verteilten Systemen.

Nachteile

- schwer zu implementieren
 - hoher Kommunikationsaufwand
 - Verfügbarkeit leidet bei Ausfällen oder Partitionen
-

11. Wann man Linearisierbarkeit braucht

Typische Fälle:

- **Locking**
 - **Leader Election**
 - **globale Constraints**
 - **Eindeutigkeitsgarantien**
 - **Koordination von Nachrichten aus mehreren Kanälen**
-

12. Eventual Consistency

Motivation

Linearisierbarkeit ist einfach zu verstehen, aber teuer:

- viele Nachrichten
- hohe Latenz
- schlechtere Skalierbarkeit
- bei fehlendem Quorum kommt das System nicht weiter

Deshalb verwenden viele Systeme schwächere Modelle wie **Eventual Consistency**.

Grundidee

Replika arbeiten zunächst mit ihrem **lokalen Zustand**.

Wenn irgendwann keine neuen Updates mehr kommen, **können** sie sich angleichen.

Es gibt aber keine feste Garantie, wann genau das passiert.

Strong Eventual Consistency

Die Folien nennen dafür zwei Kerneigenschaften:

Eventual Delivery

Jedes Update auf einem funktionierenden Replika erreicht irgendwann alle anderen funktionierenden Replika.

Convergence

Zwei Replika mit derselben Menge an Updates enden im selben Zustand, auch wenn die Reihenfolge der Verarbeitung unterschiedlich war.

Eigenschaften

- kein Warten auf Netzwerk-Reparatur nötig
 - kausaler Broadcast oder etwas Schwächeres reicht oft
 - bei nebenläufigen Updates können Konflikte entstehen
 - diese müssen aufgelöst werden, z. B. mit **Last Writer Wins**
-

13. CAP-Theorem

Die drei Eigenschaften

Consistency

Hier gemeint im Sinne von **Linearisierbarkeit**:

Nach einem Update sehen Leser dieselben aktuellen Daten.

Availability

Jeder Request an einen funktionierenden Knoten bekommt irgendwann eine Antwort.

Partition Tolerance

Das System funktioniert auch dann weiter, wenn das Netzwerk in getrennte Teile zerfällt.

Aussage des CAP-Theorems

Ein verteiltes System kann nicht gleichzeitig alle drei Eigenschaften vollständig haben.

Praktisch heißt das:

Bei einer Netzwerkpartition muss man sich zwischen

- stärkerer Konsistenz
- und höherer Verfügbarkeit

entscheiden.

Intuition

Wenn ein Wert auf Knoten 1 geschrieben wird, aber wegen einer Partition nicht rechtzeitig zu Knoten 2 repliziert werden kann, dann gibt es nur zwei Möglichkeiten:

- Lesen auf Knoten 2 blockieren → **Konsistenz**
 - alten Wert zurückgeben → **Verfügbarkeit**
-

Praxis

Ein einzelner Server ist typischerweise **CA**.

Ein verteiltes System muss meist **Partition Tolerance** annehmen.

Deshalb gibt es in der Praxis fast immer einen Trade-off zwischen Konsistenz und Verfügbarkeit.

14. PACELC

PACELC erweitert CAP um den Fall **ohne Partition**.

Frage:

- Bei Partition: Consistency oder Availability?
- Sonst: Consistency oder Latency?

Damit wird klar: Auch ohne Fehler muss man oft zwischen starker Konsistenz und niedriger Latenz abwägen.

15. ACID vs. BASE

ACID

Klassisches starkes Modell:

- Atomicity
- Consistency
- Isolation
- Durability

BASE

Pragmatischer Ansatz vieler NoSQL-Systeme:

- Basically Available
- Soft State
- Eventually Consistent

Soft State

Das System antwortet schon, obwohl Änderungen noch nicht überall angekommen sind.

16. Kausale Konsistenz

Idee

Ein System ist **kausal konsistent**, wenn es die kausale Reihenfolge von Ereignissen respektiert:

- Ursache vor Wirkung

- Senden vor Empfangen
- Frage vor Antwort

Beispiel

Wenn ein Client einen Wert liest, sollte er auch die Werte sehen können, die kausal davor liegen.

Die Folien nennen **Snapshot Isolation** als Beispiel einer solchen Umsetzung.

17. Kausale Konsistenz vs. Linearisierbarkeit

Linearisierbarkeit

- totale Ordnung
- jede Operation wirkt atomar
- stark, aber teuer

Kausale Konsistenz

- nur partielle Ordnung
- nur kausal abhängige Ereignisse müssen geordnet sein
- nebenläufige Operationen müssen nicht global sortiert werden

Wichtige Aussage

Kausale Konsistenz ist laut Folien die stärkste Konsistenzform, die noch ohne starke Latenzstrafe auskommt und bei Netzwerkpartitionen verfügbar bleiben kann.

18. Einordnung der Modelle

Von stark nach schwächer ungefähr:

- Strict Serializable
- Serializable
- Linearizable
- Sequential
- Causal
- PRAM / FIFO-artige Modelle
- Read-your-writes / Monotonic Reads / ähnliche Client-Garantien

Die Grafik auf der vorletzten Folie zeigt zusätzlich:

- starke Modelle sind bei Netzfehlern oft nicht vollständig verfügbar
 - schwächere Modelle sind verfügbarer, geben aber weniger Ordnungsgarantien
-

19. Minimale Systemanforderungen laut Folie

Die letzte Folie ordnet Probleme grob den nötigen Annahmen zu:

- **Strict serializable** → alle beteiligten Knoten, partiell synchron
 - **Atomic commit** → alle beteiligten Knoten, partiell synchron
 - **Consensus / Total Order Broadcast / linearizable CAS** → Quorum, partiell synchron
 - **linearizable get / set** → Quorum, asynchron
 - **eventual consistency / causal broadcast / FIFO broadcast** → nur lokale Replika, asynchron
-

20. Kurzfazit

Verteilte Transaktionen

Sorgen dafür, dass Änderungen über mehrere Systeme atomar ablaufen.

2PC

Klassischer Ansatz, aber blockierend bei Koordinator-Ausfall.

Linearisierbarkeit

Sehr starkes Modell: jede Operation sieht aus wie auf einer einzigen Kopie.

Strict Serializability

Noch stärker: Linearisierbarkeit plus Serialisierbarkeit.

Eventual Consistency

Schwächer, aber oft besser für Skalierbarkeit und Verfügbarkeit.

CAP

Bei Partitionen muss man praktisch zwischen Konsistenz und Verfügbarkeit abwägen.

Kausale Konsistenz

Oft ein guter Mittelweg: stärker als viele schwache Modelle, aber günstiger als Linearisierbarkeit.

21. Merksätze

- Konsistenz ist nicht gleich Konsistenz: der Begriff hängt vom Kontext ab.
- 2PC löst Atomizität, aber nicht elegant die Verfügbarkeitsprobleme.
- Quoren allein garantieren noch keine Linearisierbarkeit.
- ABD macht Quorum-Reads und -Writes linearisierbar.
- Strict Serializability ist sehr stark, aber teuer.
- Eventual Consistency ist oft die pragmatische Wahl großer verteilter Systeme.
- CAP heißt nicht: wähle immer nur zwei. Es heißt: bei Partitionen gibt es einen harten Trade-off.
- Kausale Konsistenz ordnet nur, was wirklich kausal zusammenhängt.